

HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 8
February 19, 2026

HOMework 2

 CS-2050 / homework-2 

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

 **homework-2** Internal template [Edit Pins](#) [Watch 0](#) [Fork 0](#)

[main](#) [1 Branch](#) [0 Tags](#) [Add file](#) [Code](#)

 **laz380 and Chuck Witt** public release.

b5effe · 4 days ago  1 Commits

 question-1	public release.	4 days ago
 question-2	public release.	4 days ago
 question-3	public release.	4 days ago
 question-4	public release.	4 days ago
 question-5	public release.	4 days ago
 question-6	public release.	4 days ago
 question-7	public release.	4 days ago
 .gitignore	public release.	4 days ago
 README.md	public release.	4 days ago

CLUSTER A BIT SLUGGISH RECENTLY



HARVARD UNIVERSITY
Information Technology

OnDemand provides an integrated, single access point for all of your HPC resources.

Latest News for HUIT Open OnDemand

Current status: Slow performance

We are aware of ongoing performance issues with HUIT Open OnDemand. You may notice:

- The Dashboard is slow to refresh or load
- Interactive Apps (including new sessions) can be slow to start or display

Our team is investigating and working to restore normal performance.



CS-2050 / lecture-examples

Q Type to search



Code



Issues



Pull requests



Actions



Projects



Wiki



Security



Insights



Settings



lecture-examples

Internal



Edit Pins



Watch

0



main



1 Branch



0 Tags



Go to file



Code



wcw398

748bde7 · 15 minutes ago



8 Commits



lecture-02

add lecture-02.

3 weeks ago



lecture-04

lecture-04 examples.

2 weeks ago



lecture-06

.

last week



lecture-07

.

2 days ago



lecture-08

.

15 minutes ago



README.md

.

2 days ago



README

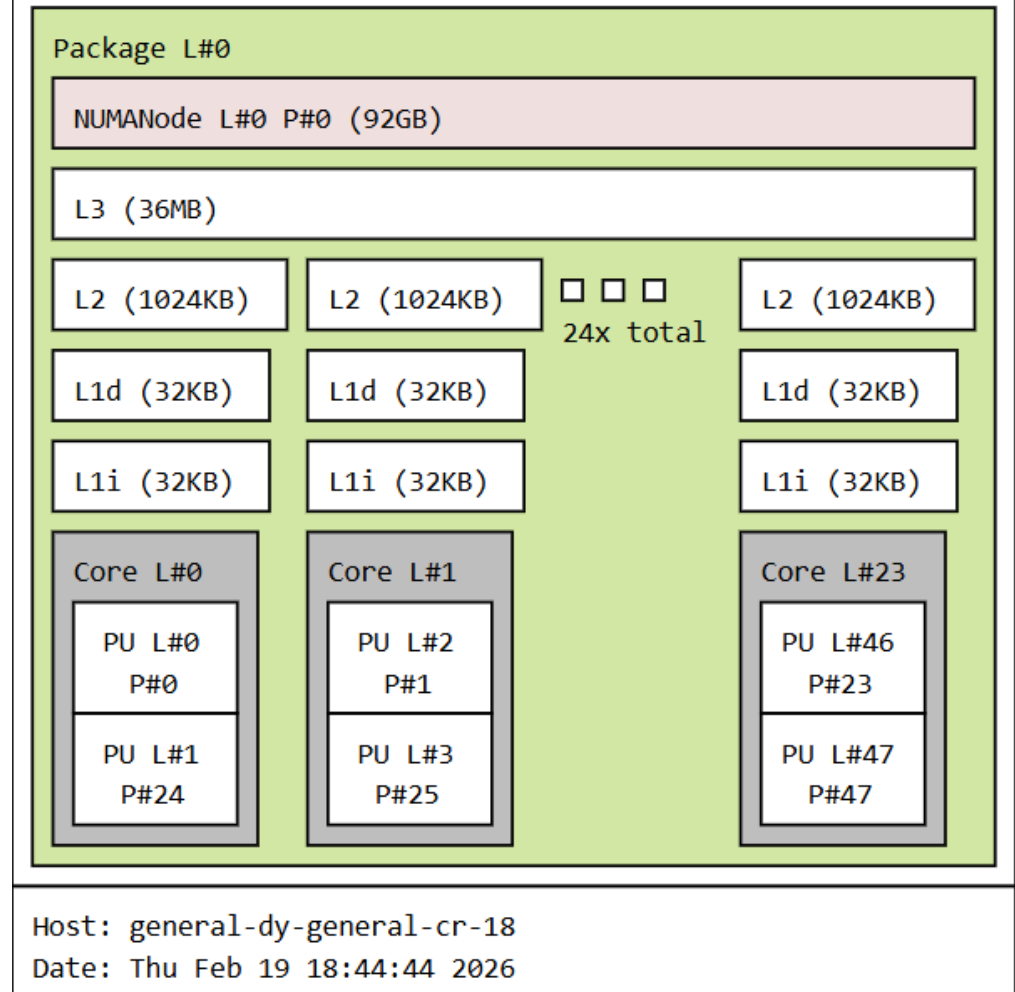


Useful Commands

example-1

HARDWARE VS SLURM

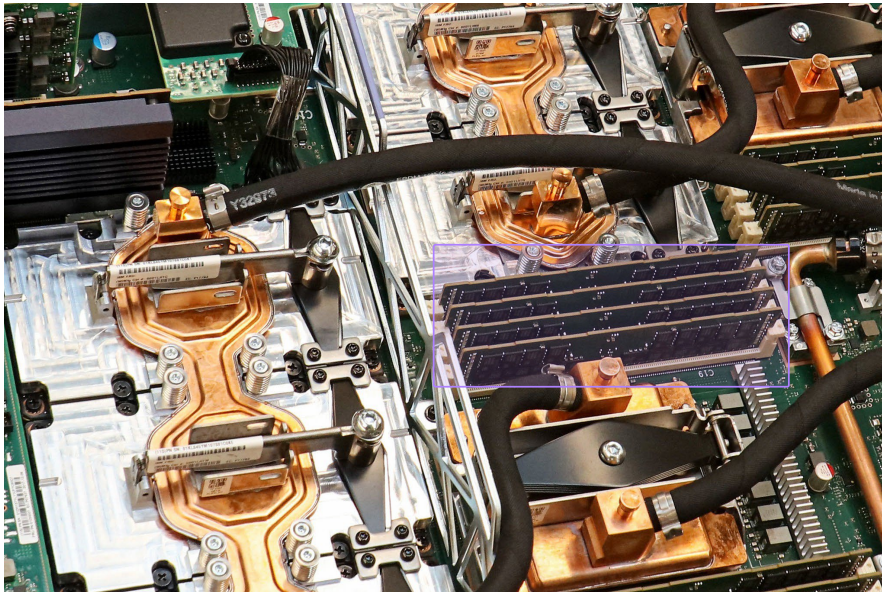
Machine (92GB total)



```
scontrol show node general-dy-general-cr-1
```

WHY DISTRIBUTED PARALLELISM?

- Maximum memory per node is limited
- For large computing tasks, single-node memory is simply not enough
 - fluid dynamics
 - molecular dynamics
 - machine learning
 - large scale data processing



- **Solution:** Many nodes connected by (HPC) network
- Memory is *distributed* among connected nodes. Communication achieved by *sending* or *receiving* messages over the network.

RECALL: PROCESSES AND THREADS

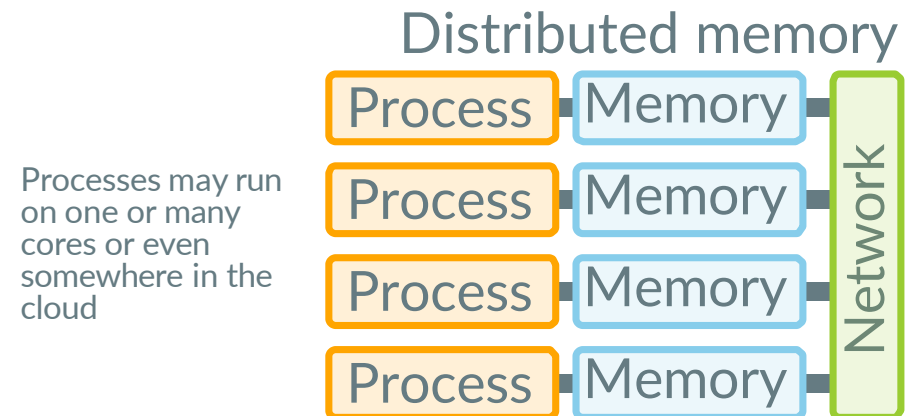
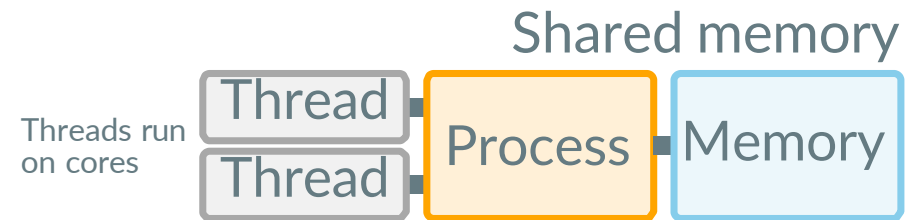
Processes:

OS creates processes - expensive to create. Each process has its own virtual address space.

Logical (software) threads:

Threads share the resources of a process. Less expensive to create.

- Does *not* mean each process must live in a separate physical memory space (or on different nodes)
- For example, many processes run simultaneously on your laptop
Does not matter whether these processes originate from individual programs or from a distributed memory code



HELLO MPI

Hello MPI:

```
1 #include <iostream>
2 #include <mpi.h>
3
4 int main(int argc, char *argv[])
5 {
6     MPI_Init(&argc, &argv);
7
8     std::cout << "Hello MPI\n";
9
10    MPI_Finalize();
11    return 0;
12 }
```

MPI compiler wrapper for C++:

```
1 $ mpic++ -show # output shortened
2 g++ -pthread -L/usr/lib/openmpi -lmpi
```

Compilation is similar as with g++:

```
1 $ mpic++ -o hello_mpi hello_mpi.cpp
```

MPI runtime wrapper:

```
1 $ mpirun -n 2 ./hello_mpi
2 Hello MPI
3 Hello MPI
```

- The `-n 2` argument creates 2 copies of the program.
- We can check that this creates 2 processes:

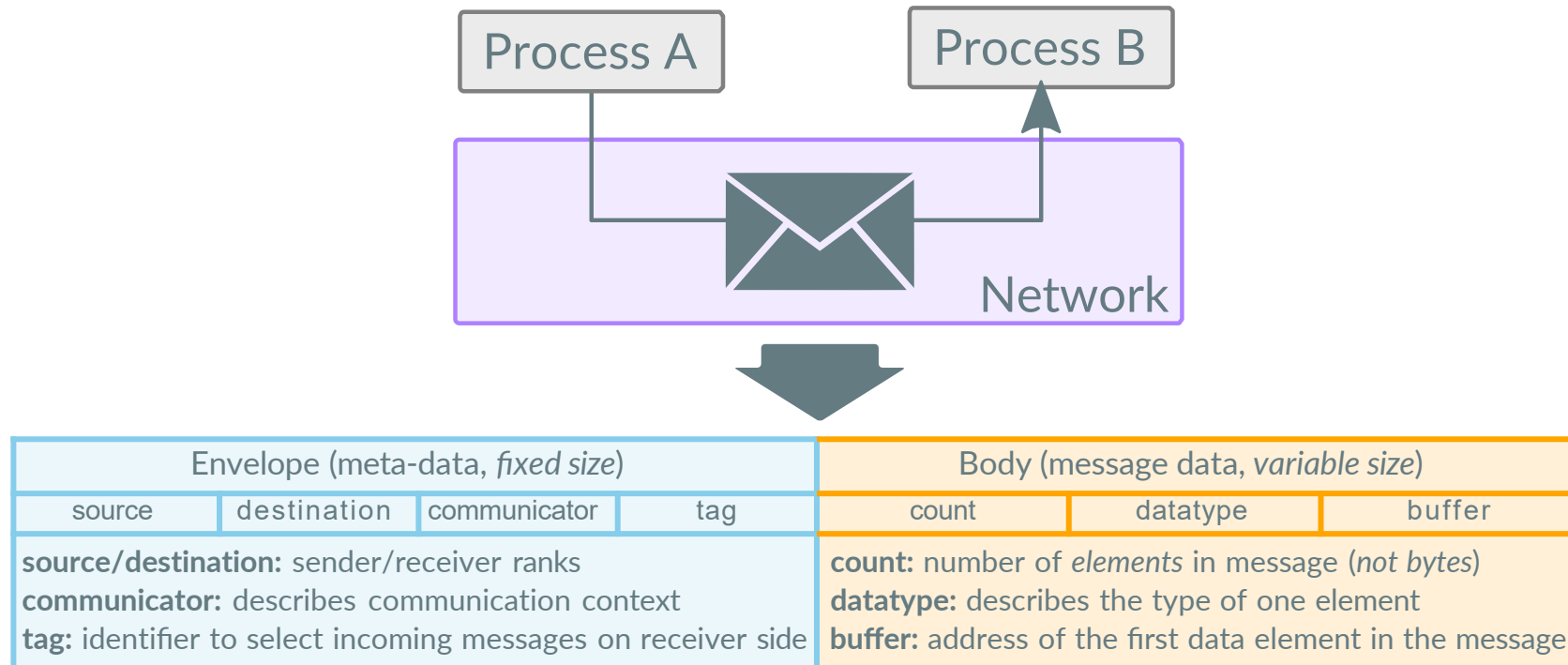
```
1 $ ps aux | grep hello_mpi
2 fabs 584178 20:00 0:00 mpirun -n 2 ./hello_mpi
3 fabs 584180 20:00 0:00 ./hello_mpi
4 fabs 584181 20:00 0:00 ./hello_mpi
```

Running MPI on SLURM:

```
$ srun -n2 -c1 ./hello_mpi
Hello MPI
Hello MPI
```

- SLURM uses `srun` to run parallel jobs.
- You have already used it in your job scripts. The option `-n2` requests 2 tasks (`--ntasks`) where each task has `-c1` cores to run on (`--cpus-per-task`).

ANATOMY OF AN MPI MESSAGE



Example:

```
1 if (Process_A) {  
2     // destination = Process B  
3     MPI_Send(send_buffer, count, datatype, destination, tag, MPI_COMM_WORLD);  
4 } else if (Process_B) {  
5     // source = Process A  
6     MPI_Recv(recv_buffer, count, datatype, source, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
7 }
```

BLOCKING POINT-TO-POINT COMMUNICATION

example-2

- Standard (blocking) sending and receiving of messages in MPI is achieved with `MPI_Send` and `MPI_Recv`, respectively.
- They are both *non-local* procedures. An `MPI_Send` *requires* a matching `MPI_Recv` to complete and vice versa.

```
1 int MPI_Send(  
2     const void *buf,          // message body  
3     int count,                // message body  
4     MPI_Datatype datatype,    // message body  
5     int dest,                 // message envelope  
6     int tag,                  // message envelope  
7     MPI_Comm comm             // message envelope  
8 )
```

```
1 int MPI_Recv(  
2     void *buf,                // message body  
3     int count,                // message body  
4     MPI_Datatype datatype,    // message body  
5     int source,               // message envelope  
6     int tag,                  // message envelope  
7     MPI_Comm comm,            // message envelope  
8     MPI_Status *status  
9 )
```

- An `MPI_Send` **matches** an `MPI_Recv` *only if* `dest`, `source`, `tag` and `comm` match.
- The return value is an error code. See `man MPI` for a full list of the codes. If the procedure is successful, it returns `MPI_SUCCESS`.
- To receive messages with arbitrary tags use `MPI_ANY_TAG` and to receive from any source use `MPI_ANY_SOURCE`.
- If the tag or source is unknown, use the `status` object to query the meta data.
- Often the status is not needed, use `MPI_STATUS_IGNORE` in this case.

MPI BARRIER

```
1  #include <mpi.h>
2
3  int main(int argc, char *argv[])
4  {
5      int rank;
6      MPI_Init(&argc, &argv);
7      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
8
9      // some code
10
11     if (0 == rank) {
12         // all ranks must call this procedure
13         MPI_Barrier(MPI_COMM_WORLD);
14     }
15     // Deadlock: other ranks do not call
16     //           MPI_Barrier
17
18     // some other code
19
20     MPI_Finalize();
21     return 0;
22 }
```

- A barrier *synchronizes* all processes in a communicator
- The procedure completes once *all processes have executed the routine*
- Your code will deadlock if not all ranks in a group participate



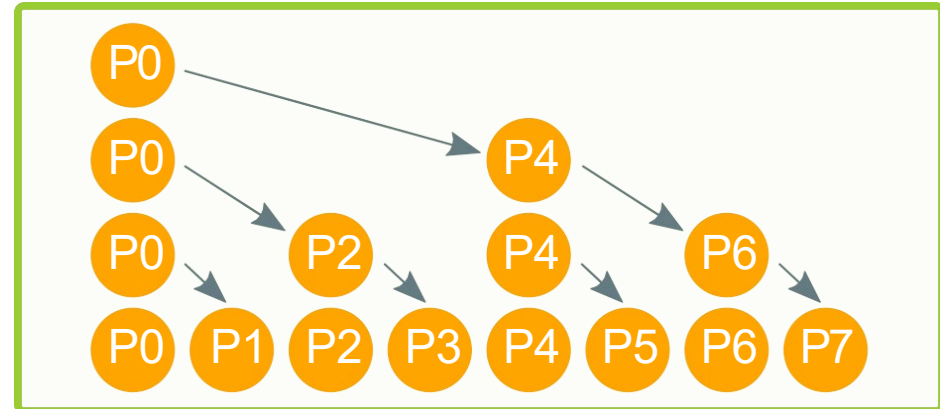
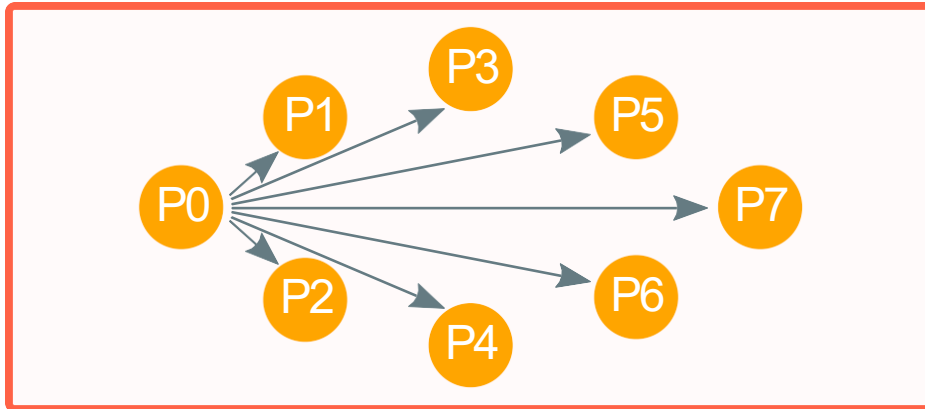
BLOCKING COLLECTIVE COMMUNICATION

Why do we have collective procedures in the API:

Example: MPI_Bcast (broadcast) — the root rank distributes data to all other ranks

Naive: the root rank must deal with 7 MPI_Send calls.
This will scale badly if many ranks are involved.

Log-step: in a tree based algorithm, root must deal only with $\log_2(n)$ MPI_Send calls, where n is the total number of ranks in the communicator. *Here other ranks help out broadcasting the data.*



Note:

- For **8 ranks**: 3 versus 7 MPI_Send calls *may not* feel like a big improvement.
- For **1024 ranks**: 10 versus 1023 MPI_Send calls is a **100x** improvement!

MPI BROADCAST

A broadcast sends data from the root rank (not necessarily rank 0) to all ranks in the group, including itself.

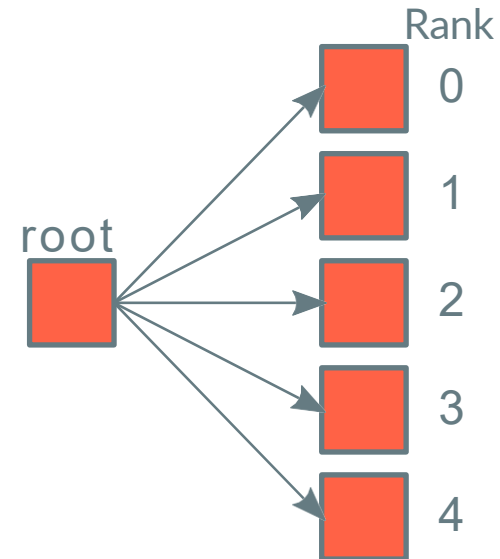
```
1 int MPI_Bcast(void *buffer,  
2             int count,  
3             MPI_Datatype datatype,  
4             int root,  
5             MPI_Comm comm)
```

A naive implementation to broadcast a message
(inefficient, don't do this)

```
1 #include <mpi.h>  
2  
3 void bcast(void *buffer, int count, MPI_Datatype datatype,  
4           int root, MPI_Comm comm)  
5 {  
6     int rank, size;  
7     MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
8     MPI_Comm_size(MPI_COMM_WORLD, &size);  
9  
10    if (rank == root) {  
11        for (int i = 0; i < size; ++i) {  
12            if (i == rank) {  
13                continue  
14            }  
15            MPI_Send(buffer, count, datatype, i, 123, comm);  
16        }  
17    } else {  
18        MPI_Recv(buffer, count, datatype, root, 123, comm,  
19               MPI_STATUS_IGNORE);  
20    }  
21 }
```

MPI internally uses a tree structure to achieve a more efficient implementation.

Example visualization for an MPI_Bcast:



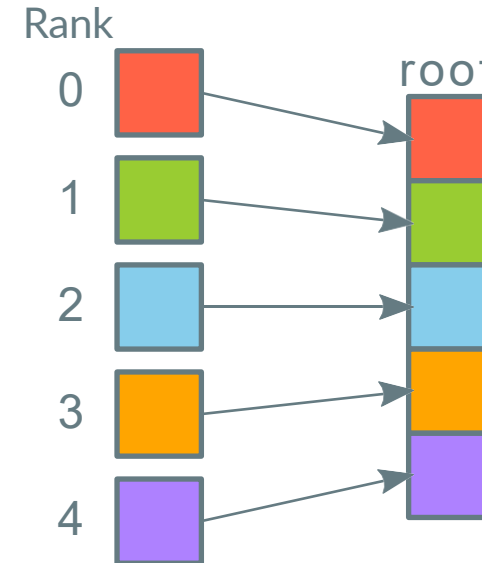
MPI GATHER

In a gather operation, each rank sends data to the root rank. The root process receives and stores the messages in *rank order*.

```
1 int MPI_Gather(const void *sendbuf,  
2             int sendcount,  
3             MPI_Datatype sendtype,  
4             void *recvbuf,  
5             int recvcount,  
6             MPI_Datatype recvtype,  
7             int root,  
8             MPI_Comm comm)
```

- The `recvbuf`, `recvcount` and `recvtype` arguments are only significant for the root process.
- The `recvcount` argument denotes the number of elements to be received from any other process.
- If there are n ranks, then the total number of elements in the receive buffer is given by $n * \text{recvcount}$. The n messages are concatenated in *rank order*.

Example visualization for an MPI_Gather:



- See Section 6.5 in the MPI 4.0 standard.
- There is a more flexible version that allows to receive a different number of elements from each process. See `man MPI_Gatherv`

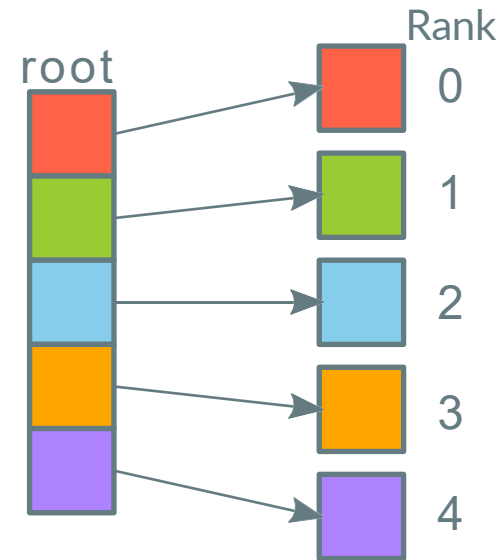
MPI SCATTER

The scatter operation is the *inverse* of the gather operation.

```
1 int MPI_Gather(const void *sendbuf,  
2               int sendcount,  
3               MPI_Datatype sendtype,  
4               void *recvbuf,  
5               int recvcount,  
6               MPI_Datatype recvtype,  
7               int root,  
8               MPI_Comm comm)
```

- The sendbuf, sendcount and sendtype arguments are only significant for the root process.
- The **sendcount** argument denotes the number of elements to be sent to any other process.
- If there are n ranks, then the total number of elements in the send buffer is given by $n * \text{sendcount}$. The i -th process with rank i receives sendcount elements starting from position $i * \text{sendcount}$ in the sendbuf buffer on the root rank.

Example visualization for an MPI_Scatter:



- See Section 6.6 in the MPI 4.0 standard.
- There is a more flexible version that allows to send a different number of elements to each process. See `man MPI_Scatterv`

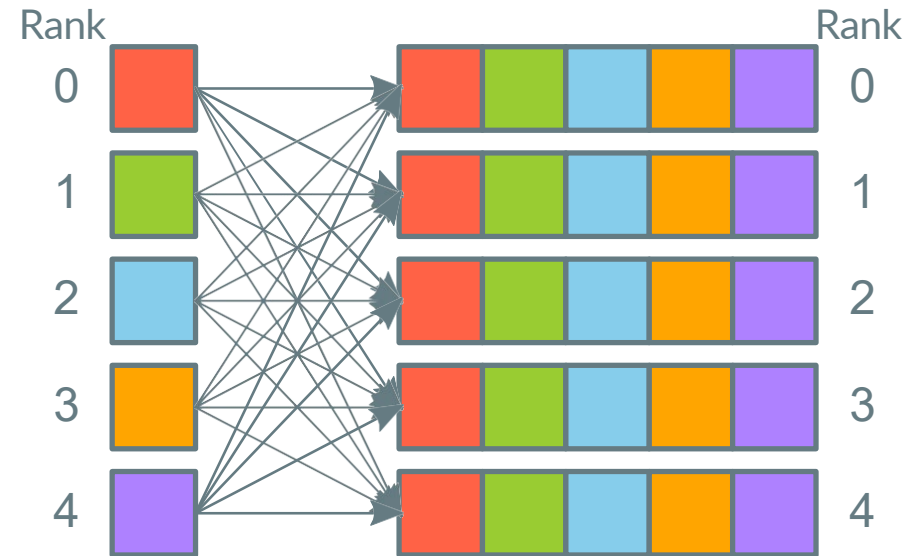
MPI GATHER TO ALL

The same as the gather operation but *all* processes receive the results instead of just root.

```
1 int MPI_Allgather(const void *sendbuf,  
2                 int sendcount,  
3                 MPI_Datatype sendtype,  
4                 void *recvbuf,  
5                 int recvcount,  
6                 MPI_Datatype recvtype,  
7                 MPI_Comm comm)
```

- The **recvcount** argument denotes the number of elements to be received from any other process.
- If there are n ranks, then the total number of elements in the receive buffer is given by $n * \text{recvcount}$. The n messages are concatenated in **rank order**.

Example visualization for an MPI_Allgather:



See Section 6.7 in the MPI 4.0 standard.

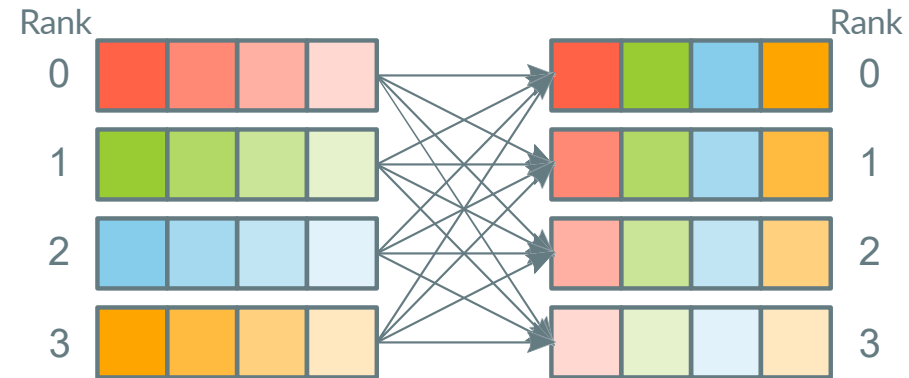
MPI ALL-TO-ALL

The all-to-all operation is an extension of the gather to all operation to the case where each process sends distinct data to each of the receivers.

```
1 int MPI_Alltoall(const void *sendbuf,  
2                 int sendcount,  
3                 MPI_Datatype sendtype,  
4                 void *recvbuf,  
5                 int recvcount,  
6                 MPI_Datatype recvtype,  
7                 MPI_Comm comm)
```

- Each process performs $2n$ independent point-to-point communications (including itself).
- Each process breaks up its local sendbuf into n blocks (each with sendcount elements). The recvbuf is similarly divided.
- Process j sends the k -th block to process k which places the data in the j -th block of its local recvbuf buffer.
- The amount of data sent must be equal to the amount of data received (pairwise between every pair of processes).

Example visualization for an MPI_Alltoall:



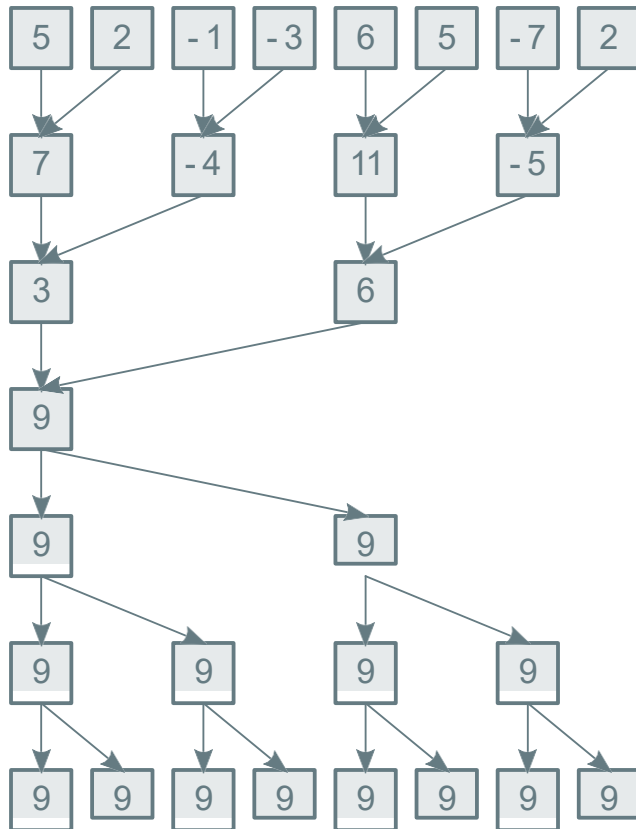
- The result is a **transpose**.
- The MPI_Alltoall is heavily used in the parallel implementation of the Fast-Fourier-Transform ([FFTW library](#)).
- This operation supports the **MPI_IN_PLACE** handle for in-place memory operations.
- See Section 6.8 in the MPI 4.0 standard.

MPI ALL-REDUCE

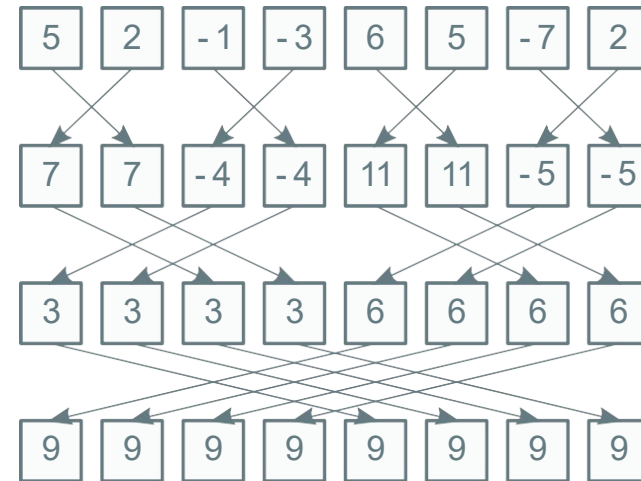
Same as MPI_Reduce but result is broadcast to all processes.

```
1 int MPI_Allreduce(const void *sendbuf, void *recvbuf, int count,  
2 MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

Naive tree implementation:



Optimized implementation:



MPI INCLUSIVE SCAN (PARTIAL REDUCTION)

For the *inclusive scan* operation, the result on rank i is the partial reduction with data from ranks $k \leq i$ (*inclusive*).

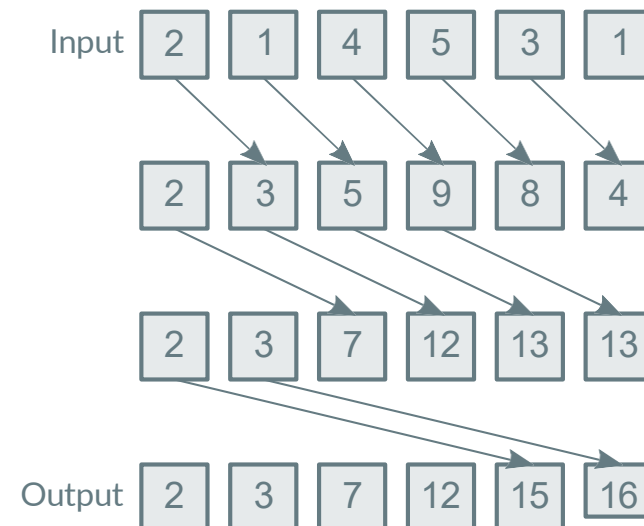
```
1 int MPI_Scan(const void *sendbuf,  
2             void *recvbuf,  
3             int count,  
4             MPI_Datatype datatype,  
5             MPI_Op op,  
6             MPI_Comm comm)
```

- The MPI_Op reduction operator can be any of the ones used for MPI_Reduce.
- You can use the MPI_IN_PLACE handle for the sendbuf argument. In this case the input is taken from the recvbuf argument and output replaces this value.

Example:

$$y_i = \sum_{k=0}^{i-1} x_k$$

Rank	0	1	2	3	4	5
Input	2	1	4	5	3	1
Output	2	3	7	12	15	16



MPI EXCLUSIVE SCAN (PARTIAL REDUCTION)

The *exclusive scan* operation is similar to the inclusive scan, except that the partial reduction does not include the data on rank i , that is

$$k < i \text{ (exclusive).}$$

```
1 int MPI_Exscan(const void *sendbuf,  
2               void *recvbuf,  
3               int count,  
4               MPI_Datatype datatype,  
5               MPI_Op op,  
6               MPI_Comm comm)
```

- The MPI_Op reduction operator can be any of the ones used for MPI_Reduce.
- You can use the MPI_IN_PLACE handle for the sendbuf argument. In this case the input is taken from the recvbuf argument and output replaces this value.

Example:

$$y_i = \sum_{k=0}^{i-1} x_k$$

Rank	0	1	2	3	4	5
Input	2	1	4	5	3	1
Output	0	2	3	7	12	15

- An example usage for an MPI_Exscan is to compute the file offset when you write compressed data into a file using parallel I/O.
- The size of the compressed buffer is only known on the process that performed the data compression.

UNKNOWN MESSAGE SIZE

```
1 #include <cstdlib>
2 #include <iostream>
3 #include <mpi.h>
4 #include <vector>
5
6 int main(int argc, char*argv[])
7 {
8     int rank;
9     MPI_Init(&argc, &argv);
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11    srand(rank);
12
13    std::vector<double> msg;
14    if(0 == rank) {
15        msg.resize(rand() % 2048, 1.0);
16        const int size = msg.size();
17        MPI_Send(&size, 1, MPI_INT, 1, 123, MPI_COMM_WORLD);
18        MPI_Send(msg.data(), msg.size(), MPI_DOUBLE,
19                 1, 321, MPI_COMM_WORLD);
20    } else if (1 == rank) {
21        int size;
22        MPI_Status status;
23        MPI_Recv(&size, 1, MPI_INT,
24                0, 123, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
25        msg.resize(size);
26        MPI_Recv(msg.data(), msg.size(), MPI_DOUBLE,
27                0, 321, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
28        std::cout << "Rank " << rank << " received" << size
29                  << " double elements\n";
30    }
31
32    MPI_Finalize();
33    return 0;
34 }
```

- What if the message size is unknown and you must know the size of an incoming message *before* you have received the message.
- *Note: we have assumed a fixed receive buffer size of 1024 characters.*
- Sending two messages is not a good solution because of the additional overhead (especially since the first message is very small latency dominated).

PROBING AN INCOMING MESSAGE

```
1 #include <cstdlib>
2 #include <iostream>
3 #include <mpi.h>
4 #include <vector>
5
6 int main(int argc, char*argv[])
7 {
8     int rank;
9     MPI_Init(&argc, &argv);
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11    srand(rank);
12
13    std::vector<double> msg;
14    if(0 == rank) {
15        msg.resize(rand() % 2048, 1.0);
16        MPI_Send(msg.data(), msg.size(), MPI_DOUBLE,
17                1, 123, MPI_COMM_WORLD);
18    } else if (1 == rank) {
19        int size;
20        MPI_Status status;
21        MPI_Probe(0, 123, MPI_COMM_WORLD, &status);
22        MPI_Get_count(&status, MPI_DOUBLE, &size);
23        msg.resize(size);
24        MPI_Recv(msg.data(), msg.size(), MPI_DOUBLE,
25                0, 123, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26        std::cout << "Rank " << rank << " received" << size
27                << " double elements\n";
28    }
29
30    MPI_Finalize();
31    return 0;
32}
```

- The `MPI_Probe` operation allows to check for incoming messages *without retrieving them*.
- The populated `MPI_Status` can be used to obtain the count of messages elements using `MPI_Get_count` afterwards.
- Probing a message to get the message status is more efficient than actually sending (small) messages processes that need to know the incoming message size.
- See Section 3.8 in the MPI 4.0 standard.

SHIFT OPERATIONS ACROSS CHAIN OF PROCESSES

- A common communication pattern is to send a message to one rank and receive a message from another (the source and destination could also be the same).
- If blocking semantics are used, then the send and receive operations must be *ordered correctly* to *prevent cyclic dependencies* that may lead to deadlock.
- Such shift operations are often needed on a lattice. **Example:** exchanging messages on a Cartesian topology of MPI processes
- For virtual topologies, see Chapter 8 in the MPI 4.0 standard.
- See lecture codes: `mpi_cyclic_shift`

- Even ranks first send to odd ranks and odd ranks receive first.
- Once completed, the odd ranks send to even ranks.



```
1 // source and destination (periodic boundaries)
2 const int left = (rank + size - 1) % size;
3 const int right = (rank + size + 1) % size;
4
5 std::vector<double> send(100, rank), recv(100);
6 if (0 == rank % 2) { // even ranks send first
7     MPI_Send(send.data(), send.size(), MPI_DOUBLE,
8             right, 123, MPI_COMM_WORLD);
9     MPI_Recv(recv.data(), recv.size(), MPI_DOUBLE,
10            left, 321, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
11 } else {
12     MPI_Recv(recv.data(), recv.size(), MPI_DOUBLE,
13            left, 123, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
14     MPI_Send(send.data(), send.size(), MPI_DOUBLE,
15            right, 321, MPI_COMM_WORLD);
16 }
```

SHIFT OPERATIONS ACROSS CHAIN OF PROCESSES

- MPI provides a *send-receive* operation that *combines* sending and receiving to/from processes (the two processes may be the same).
- MPI will take care of the cyclic dependencies and therefore avoid deadlock.
- This operation is often used on a Cartesian topology of processes together with `MPI_Cart_shift` to determine the source and destination ranks.

```
1 int MPI_Sendrecv(const void *sendbuf,
2                 int sendcount,
3                 MPI_Datatype sendtype,
4                 int dest,
5                 int sendtag,
6                 void *recvbuf,
7                 int recvcount,
8                 MPI_Datatype recvtype,
9                 int source,
10                int recvtag,
11                MPI_Comm comm,
12                MPI_Status *status)
```

MPI send-receive operation:

```
1 // source and destination (periodic boundaries)
2 const int left = (rank + size - 1) % size;
3 const int right = (rank + size + 1) % size;
4
5 std::vector<double> send(100, rank), recv(100);
6 MPI_Sendrecv(send.data(), send.size(), MPI_DOUBLE, right, 123,
7              recv.data(), recv.size(), MPI_DOUBLE, left, 123,
8              MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Manual send and receive pairs:

```
1 // source and destination (periodic boundaries)
2 const int left = (rank + size - 1) % size;
3 const int right = (rank + size + 1) % size;
4
5 std::vector<double> send(100, rank), recv(100);
6 if (0 == rank % 2) { // even ranks send first
7     MPI_Send(send.data(), send.size(), MPI_DOUBLE,
8             right, 123, MPI_COMM_WORLD);
9     MPI_Recv(recv.data(), recv.size(), MPI_DOUBLE,
10            left, 123, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
11 } else {
12     MPI_Recv(recv.data(), recv.size(), MPI_DOUBLE,
13            left, 123, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
14     MPI_Send(send.data(), send.size(), MPI_DOUBLE,
15            right, 123, MPI_COMM_WORLD);
16 }
```

Note: there is also an `MPI_Sendrecv_replace` for which the send and receive buffer is the same (MPI will allocate internal buffer space for the operation).

NON-BLOCKING POINT-TO-POINT COMMUNICATION

- Standard (non-blocking) sending and receiving of messages in MPI is achieved with `MPI_Isend` and `MPI_Irecv`, respectively.
- They are both (*all* non-blocking procedures are) *local* and *incomplete*. The "I" in the procedure name stands for *incomplete* and *immediate*.
- A send operation requires a matching receive operation and vice versa.
- You can match non-blocking send and receive operations with blocking receive and send operations, respectively.

NON-BLOCKING POINT-TO-POINT COMMUNICATION

- Standard (non-blocking) sending and receiving of messages in MPI is achieved with `MPI_Isend` and `MPI_Irecv`, respectively.

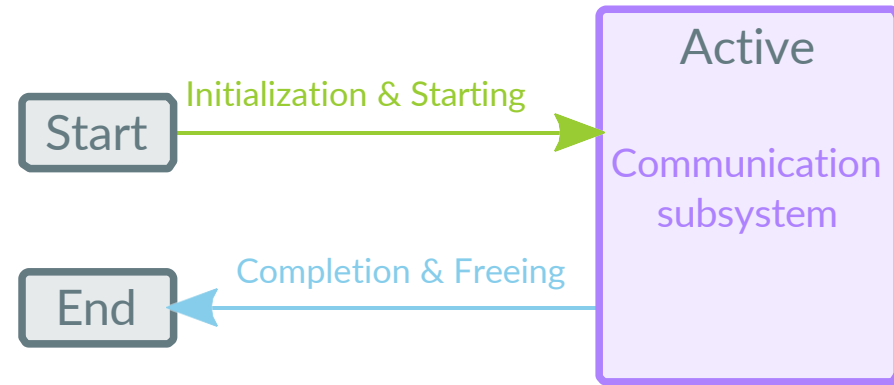
```
1 int MPI_Isend(  
2     const void *buf,           // message body  
3     int count, MPI_Datatype    // message body  
4     datatype,                 // message body  
5     int dest,                 // message envelope  
6     int tag,                 // message envelope  
7     MPI_Comm comm,           // message envelope  
8     MPI_Request *request  
9 )
```

```
1 int MPI_Irecv(  
2     void *buf,                // message body  
3     int count,                // message body  
4     MPI_Datatype datatype,    // message body  
5     int source,               // message envelope  
6     int tag,                 // message envelope  
7     MPI_Comm comm,           // message envelope  
8     MPI_Request *request  
9 )
```

- The same rules apply to the non-blocking version as for the blocking counterpart but an `MPI_Request` object is required this time for both procedures!
- The message status is obtained later by calling `MPI_Wait` or `MPI_Test` (or their variations).

COMMUNICATION COMPLETION

- All non-blocking communication procedures require a *request* object.
- The *completion* of a send or receive operation **guarantees** that the communication buffers are safe to be used again.
- The `MPI_REQUEST_NULL` designates a *null handle*.
- An `MPI_Request` object is set to `MPI_REQUEST_NULL` by `MPI_Wait` or `MPI_Test` when the operation has completed.



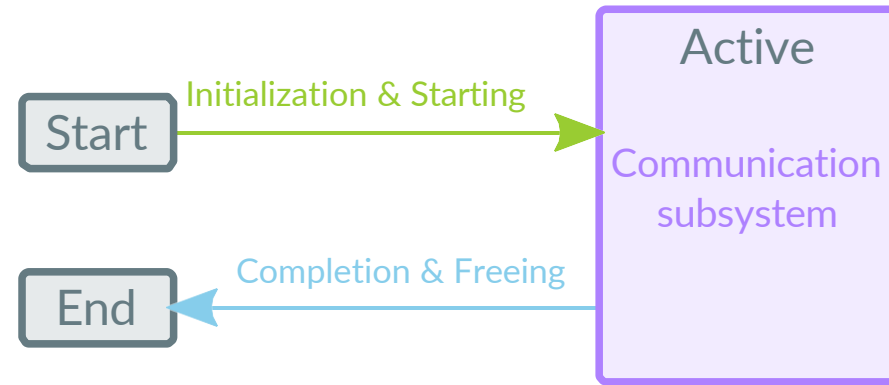
Completion guarantees that the communication buffer is safe to be used again for read/write operations.

```
1 int MPI_Isend(  
2     const void *buf,          // message body  
3     int count, MPI_Datatype // message body  
4     datatype,                // message body  
5     int dest,                // message envelope  
6     int tag,                 // message envelope  
7     MPI_Comm comm,           // message envelope  
8     MPI_Request *request  
9 )
```

```
1 int MPI_Irecv(  
2     void *buf,                // message body  
3     int count,                // message body  
4     MPI_Datatype datatype,    // message body  
5     int source,               // message envelope  
6     int tag,                  // message envelope  
7     MPI_Comm comm,           // message envelope  
8     MPI_Request *request  
9 )
```

COMMUNICATION COMPLETION

- The `MPI_REQUEST_NULL` designates a *null handle*. An `MPI_Request` object is set to `MPI_REQUEST_NULL` by `MPI_Wait` or `MPI_Test` when the operation has completed.



```
1 int MPI_Wait(MPI_Request *request,  
2             MPI_Status *status)
```

- The call to `MPI_Wait` returns when the operation identified by request is *complete*.
- request will be deallocated and set to `MPI_REQUEST_NULL` upon completion.
- This procedure is *non-local*.

```
1 int MPI_Test(MPI_Request *request, int *flag,  
2             MPI_Status *status)
```

- The call to `MPI_Test` returns `flag = true` when the operation identified by request is *complete*.
- request will be deallocated and set to `MPI_REQUEST_NULL` upon completion.
- This procedure is *local*.

NON-BLOCKING COLLECTIVES

Why non-blocking collectives?

- Performance can be improved by *overlapping* communication and computation.
- Non-blocking collective operations are optimized algorithms to exploit overlap and avoid synchronization.
- The non-blocking collective communication model is similar to the model used for non-blocking point-to-point communication.
- Non-blocking collectives require an `MPI_Request` object and are *compatible with all completion calls* we have discussed.

Unlike non-blocking point-to-point communication, non-blocking collectives *cannot be matched with a blocking collective*.

```
1 if (0 == rank) {
2     MPI_Request req;
3     MPI_Ireduce(sendbuf, recvbuf, count,
4                 datatype, op, root, comm, &req);
5 } else if (1 == rank) {
6     // Illegal code:
7     MPI_Reduce(sendbuf, recvbuf, count,
8                datatype, op, root, comm);
9 }
```

NON-BLOCKING COLLECTIVES

Prototypes of non-blocking collectives:

- The function signatures of non-blocking collectives are identical to their blocking counterparts, except that an additional MPI_Request argument is necessary.
- All non-blocking procedures are prefixed with "I" for *incomplete* and *immediate*.

```
1 int MPI_Ireduce(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype,  
2                MPI_Op op, int root, MPI_Comm comm, MPI_Request *request)
```

```
1 int MPI_Ibarrier(MPI_Comm comm, MPI_Request *request)
```

```
1 int MPI_Ibcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm,  
2                MPI_Request *request)
```

```
1 int MPI_Igather(const void *sendbuf, int sendcount, MPI_Datatype sendtype,  
2                void *recvbuf, int recvcount, MPI_Datatype recvtype,  
3                int root, MPI_Comm comm, MPI_Request *request)
```

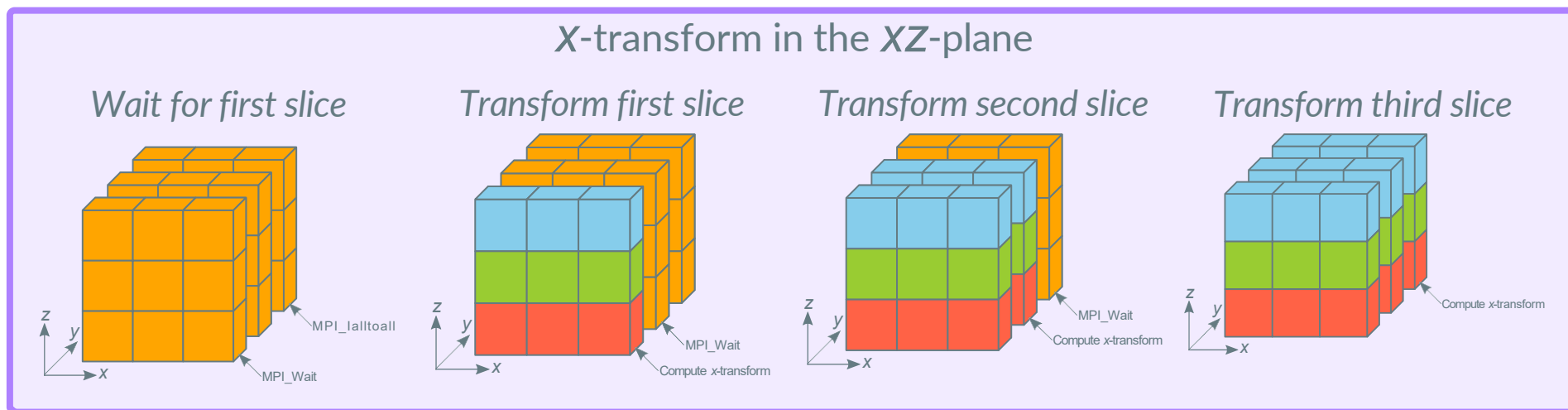
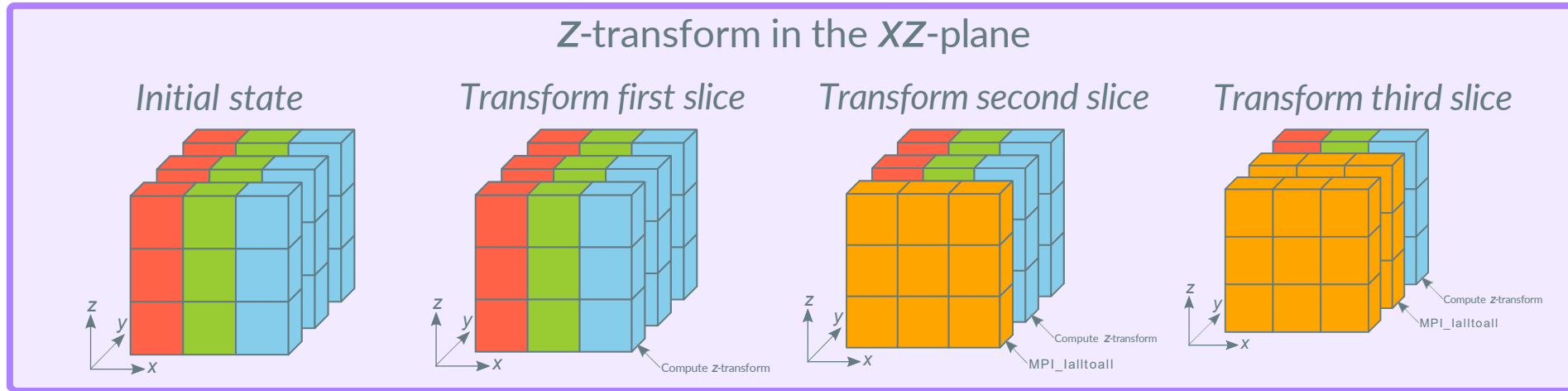
```
1 int MPI_Iscatter(const void *sendbuf, int sendcount, MPI_Datatype sendtype,  
2                 void *recvbuf, int recvcount, MPI_Datatype recvtype,  
3                 int root, MPI_Comm comm, MPI_Request *request)
```

```
1 int MPI_Ialltoall(const void *sendbuf, int sendcount, MPI_Datatype sendtype,  
2                  void *recvbuf, int recvcount, MPI_Datatype recvtype,  
3                  MPI_Comm comm, MPI_Request *request)
```

```
1 int MPI_Iexscan(const void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm  
comm, MPI_Request *request)
```

NON-BLOCKING MPI_IALLTOALL EXAMPLE

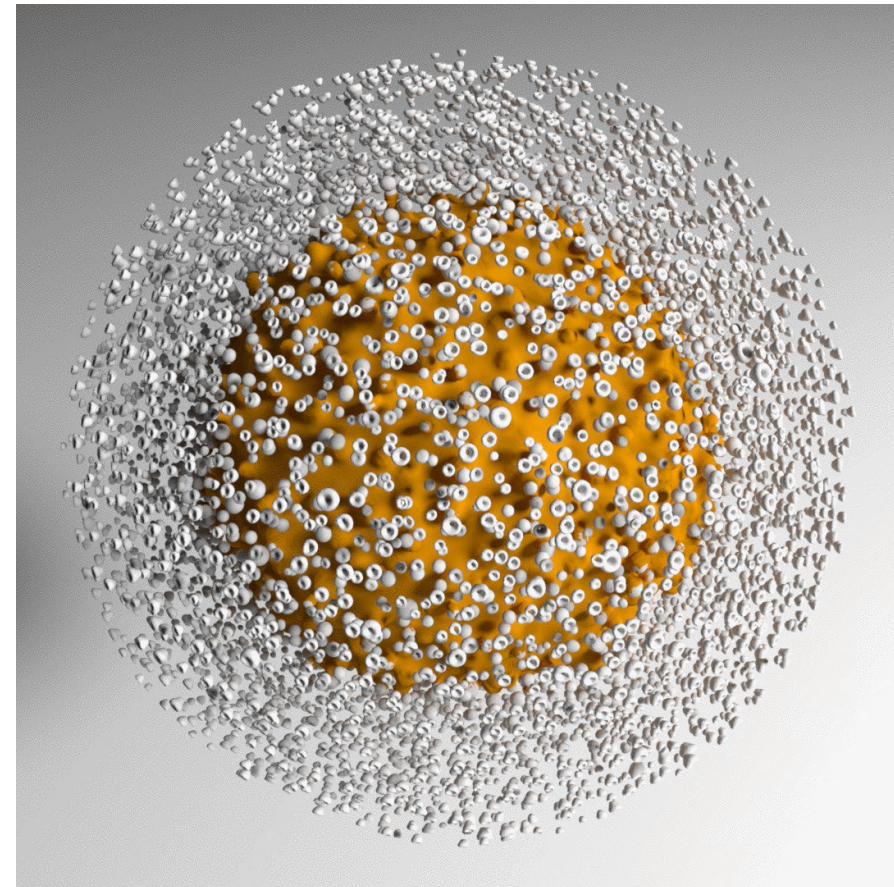
A 3D Fast-Fourier-Transform (FFT) can be carried out with three 1D transforms and transposition of the data. Assume we have already transformed along the y -coordinate.



INPUT/OUTPUT OPERATIONS

- When you develop applications, you will often need facilities to deal with input and output of data.
- **Input:** you may need to be able to *restart* your program from a previous state that you have saved in a file (checkpoints).
- **Output:** you may need to analyse your data with a third-party program for which you must *write* a certain subset of your data in a certain interval.

- Reading or writing data often (especially large amounts of data) will quickly become the bottleneck in your application. *Efficient I/O is important.*
- → Sometimes it makes sense to perform (parallel) in-situ analysis of your data instead of writing data files.



Visualization of cloud cavitation collapse with 12500 bubbles. Data for the visualization includes pressure (yellow isosurface) and gas volume fraction (bubbles) which are **884GB** of data for each field (uncompressed). Still manageable for Peta-scale simulations. With exa-scale, will be difficult to treat even larger amounts of data (file I/O).

SEND TO ROOT?

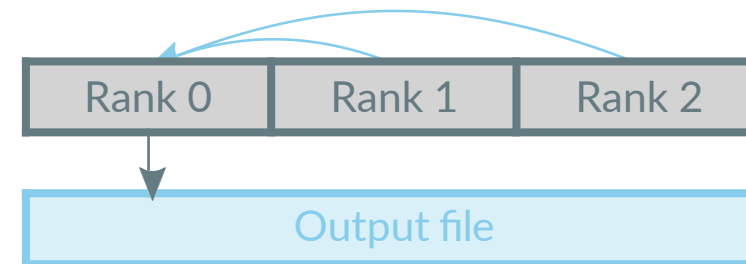
- Parallel I/O challenging

```
1 void dump_posix(const std::vector<double> &v)
2 {
3     int rank, size;
4     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5     MPI_Comm_size(MPI_COMM_WORLD, &size);
6     const int N = static_cast<int>(v.size()) - 2;
7     std::vector<double> u(size * N);
8     MPI_Gather(&v[1], N, MPI_DOUBLE,
9              u.data(), N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
10    if (0 == rank) {
11        std::ofstream out("u.dat", std::ios::app);
12        out << std::scientific;
13        for (size_t i = 0; i < u.size(); ++i) {
14            out << ' ' << u[i];
15        }
16        out << '\n';
17    }
18 }
```

- All processes send their data to the **root** rank (MPI_Gather).
- Only the *root* rank writes the data (sequential write).

Pros: one single file

Cons: not scalable!



ALTERNATIVE: EACH RANK WRITES A FILE

- To avoid the MPI_Gather, each rank writes its data to a separate file.
- Post-processing the data from many split files is very cumbersome and does not scale. You must write post-processing routines that can deal with the split file structure.
- Third-party visualization tools like ParaView will require a custom reader.

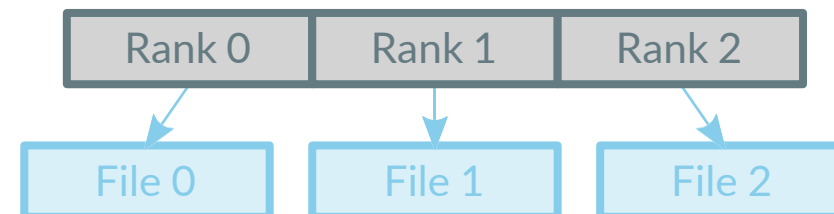
1D diffusion problem from last time

```
1 void dump_rank(const std::vector<double> &v)
2 {
3     int rank;
4     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5     const int N = static_cast<int>(v.size()) - 2;
6     std::ofstream out("u_rank" + std::to_string(rank)
7                     + ".dat", std::ios::app);
8     out << std::scientific;
9     for (int i = 1; i < N; ++i) {
10         out << ' ' << v[i];
11     }
12     out << '\n';
13 }
```

- All processes write their data in separate file (parallel).

Pros: *scalable*

Cons: *many (possibly) small files which is bad for HPC filesystems (e.g. Lustre)*



MPI I/O

- MPI provides scalable procedures for efficient parallel I/O. See Chapter 14 in the MPI 4.0 standard.

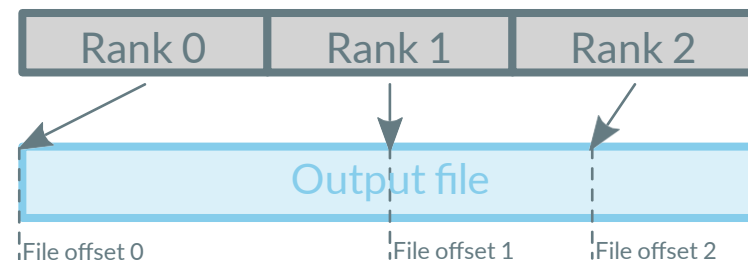
1D diffusion problem

```
1 void dump_mpi_binary(const std::vector<double> &v)
2 {
3     int rank;
4     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5     MPI_File fh; // MPI file handle
6     MPI_Offset bytes, file_offset;
7
8     const int N = static_cast<int>(v.size()) - 2;
9     bytes = N * sizeof(double);
10    file_offset = rank *
bytes; 11
12    MPI_File_open(MPI_COMM_WORLD,
13                  "u_mpi.bin",
14                  MPI_MODE_CREATE | MPI_MODE_WRONLY,
15                  MPI_INFO_NULL, &fh);
16    MPI_File_get_size(fh, &bytes); // append to file
17    file_offset += bytes;
18    MPI_File_write_at_all(fh, file_offset, v.data(),
19                          N, MPI_DOUBLE, MPI_STATUS_IGNORE);
20    MPI_File_close(&fh);
21 }
```

- All processes write their data into *the same file* in parallel.
- Think of a file as a stream of bytes → byte offsets for different ranks are now important.

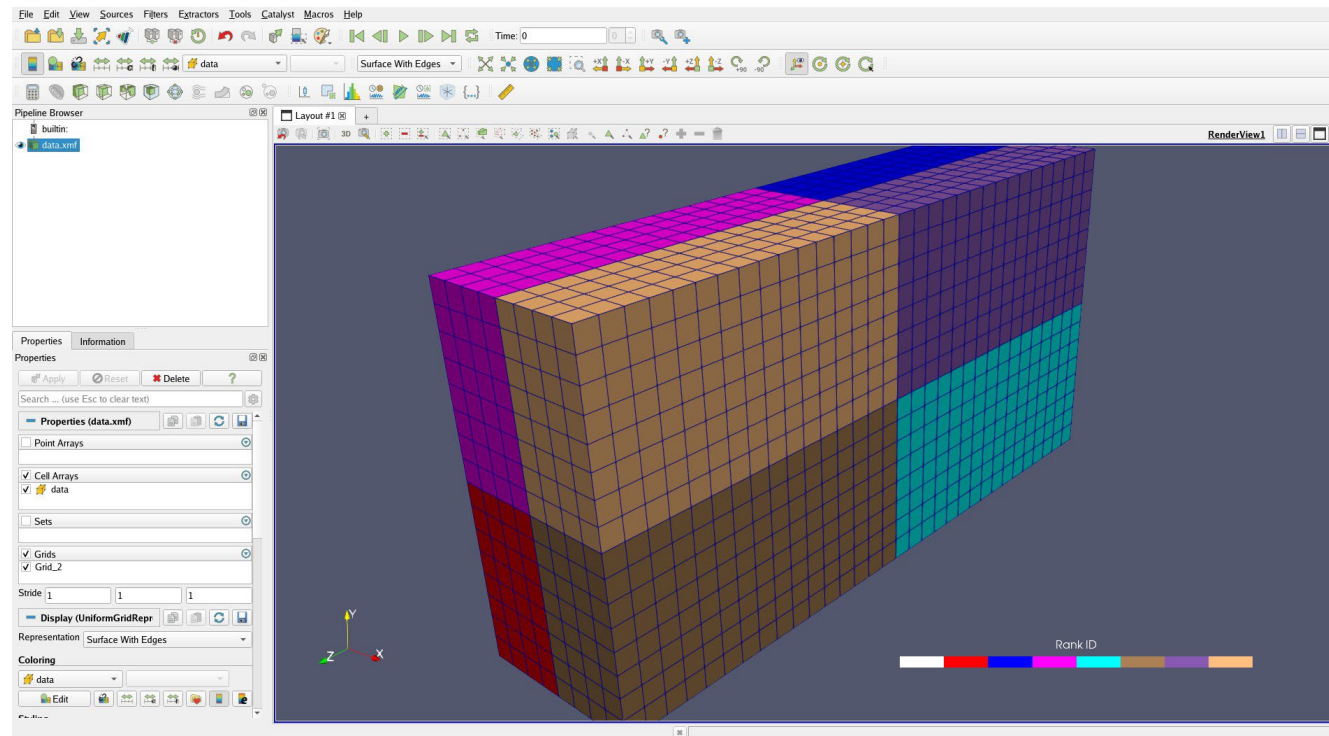
Pros: parallel and scalable

Cons: can lead to bad performance if not carefully implemented



PARALLEL I/O WITH HDF5 AND MPI

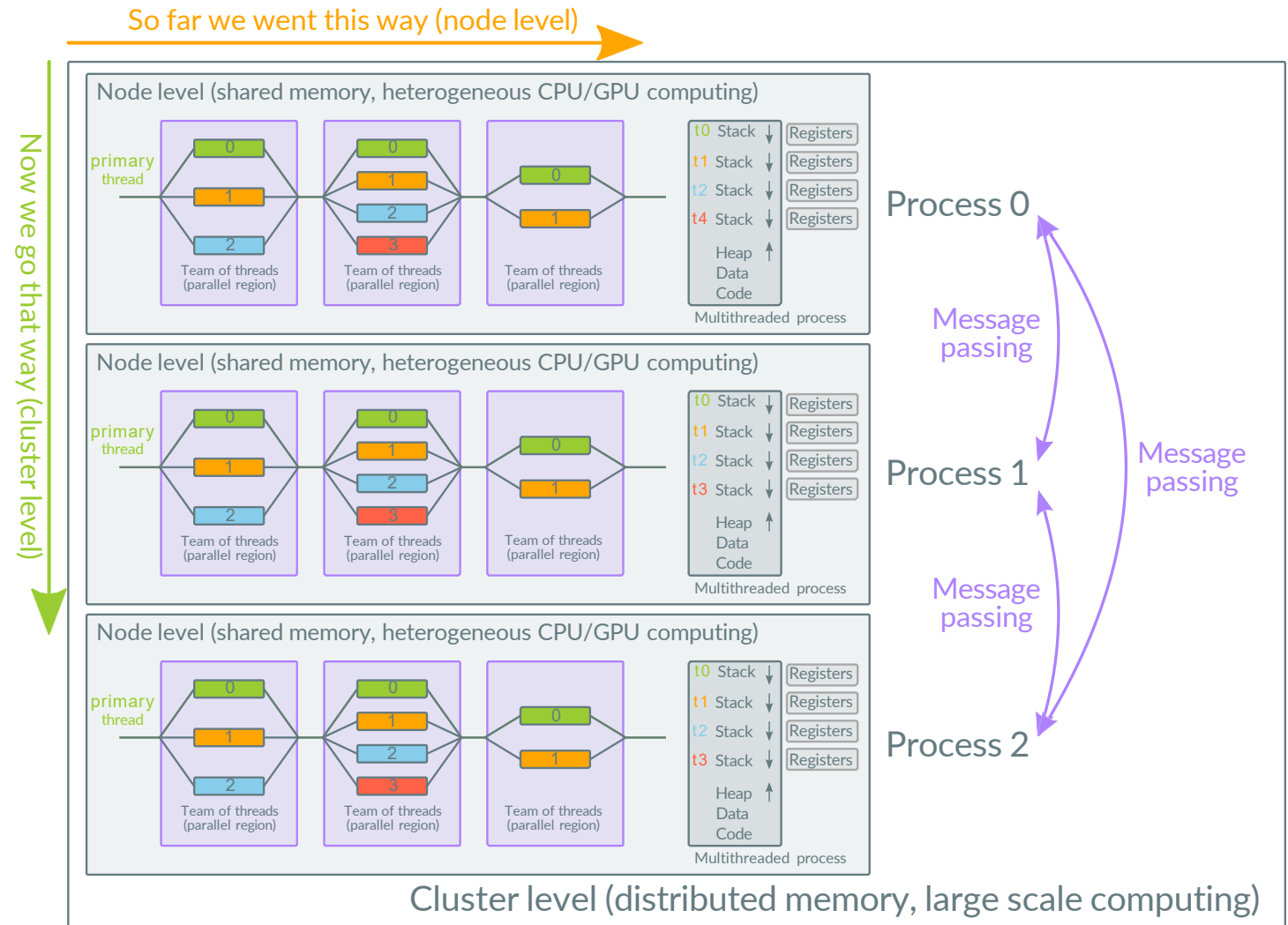
- The Hierarchical Data Format (HDF) is a widely used file format designed for large amounts of data.
- The parallel HDF5 library uses MPI and you can therefore integrate it easily in your applications.
- A python interface exists as well: <https://www.h5py.org/> (example in lecture code)
- HDF5 data can easily be visualized with Open Source tools such as ParaView:



ORTHOGONALITY

example-5

- Shared memory and distributed memory are two orthogonal programming models



SETUP

- Request a full compute node

```
srun --nodes=1 --exclusive --time=00:20:00 --pty bash
```

- Load the CS 2050 environment

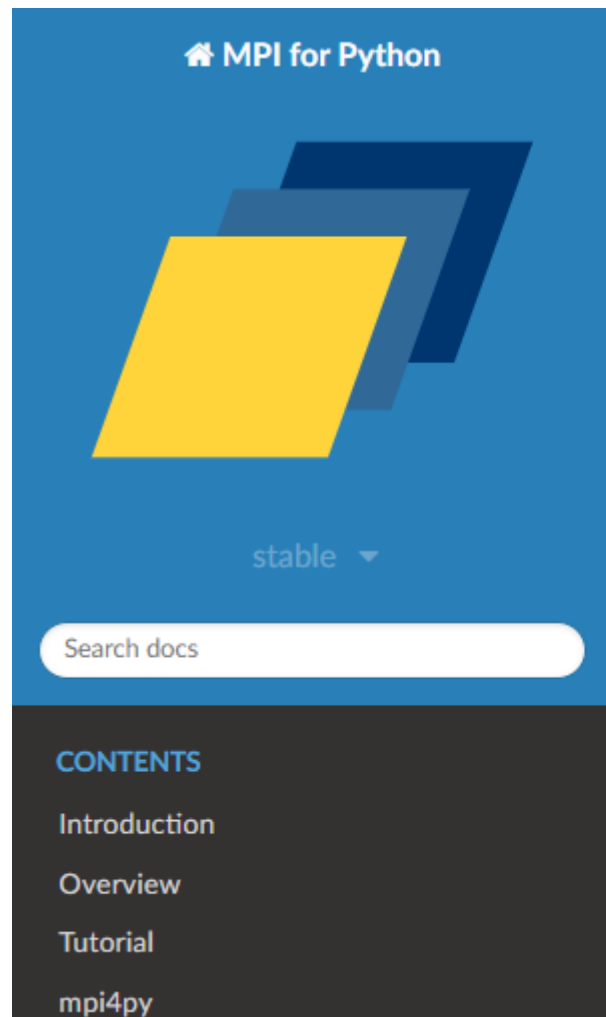
```
. ~/161588/spack/share/spack/setup-env.sh  
spack env activate -p CS-2050
```

Euler's number

This example approximates e using the formula:

$$e \approx \sum_{n=0}^{N-1} \frac{1}{n!}$$

RECALL: What makes this series
challenging to parallelize efficiently?



MPI for Python

Author: Lisandro Dalcin
Contact: dalcinl@gmail.com
Date: Oct 10, 2025

Abstract

MPI for Python provides Python bindings for the *Message Passing Interface* (MPI) standard, allowing Python applications to exploit multiple processors on workstations, clusters and supercomputers.

This package builds on the MPI specification and provides an object oriented interface resembling the MPI-2 C++ bindings. It supports point-to-point (sends, receives) and collective (broadcasts, scatters, gathers) communication of any *picklable* Python object, as well as efficient communication of Python objects exposing the Python buffer interface (e.g. NumPy arrays and builtin bytes/array/memoryview objects).

LOOKING AHEAD

PARALLEL SPEEDUP

- *Speedup* is a *fundamental* metric for parallelism.
- Speedup is a *ratio of speeds* and expresses the performance improvement as parallelism increases.
- If $T_s = T(n; 1)$ is the *sequential* execution time of a program to perform work W_s and $T_p = T(n; p)$ is the execution time when the program is run in *parallel* on p processors with work W_p , then the metric

$$S = S_p = \frac{W_p/T_p}{W_s/T_s}$$

expresses the *parallel speedup*.

- Note that the *rate* W/T is the *speed* at which work W is performed.
- In the case $W_p = W_s$ the speedup becomes a ratio of execution times:

$$S = S_p = \frac{T_s}{T_p}$$

- $S > 1$ conveys a performance *improvement* and $S < 1$ conveys a performance *degradation*.
- The *ideal* or perfect speedup is

$$S_p = p$$

and $S_p > p$ is referred to as *superlinear* speedup.

AMDAHL'S LAW

- Amdahl's law describes the *maximum expected improvement* to an overall system when only part of the system is improved. If the **problem size remains fixed** and the number of parallel processors is increased in the *parallel part* of an application, then Amdahl's law expresses the *maximum speedup theoretically possible*.

- Let f be the *serial fraction* of an application (part of code that *cannot be parallelized*). The parallel execution time for Amdahl's law is

$$T_p = f T_s + (1 - f) \frac{T_s}{p}$$

assuming negligible parallel overhead ($T_o = 0$).

The speedup for this model is given by

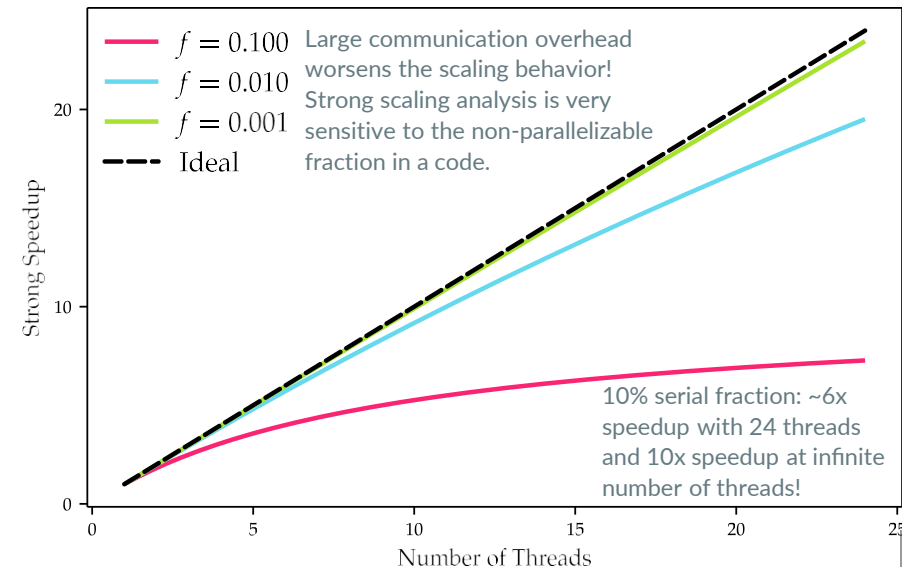
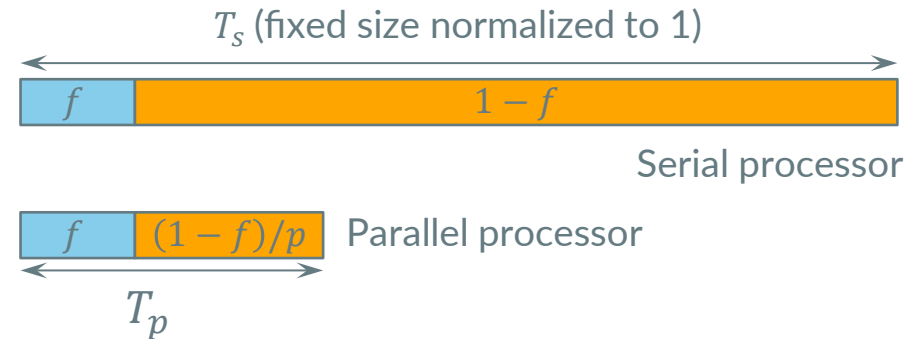
$$S_p = \frac{T_s}{T_p} = \frac{1}{f + \frac{1-f}{p}}$$

Note that the maximum speedup is bounded by the *serial fraction* since $\lim_{n \rightarrow \infty} S_p = \frac{1}{f}$.

STRONG SCALING

- A speedup analysis formulated using Amdahl's Law is called **strong scaling**.
- It assumes the problem size is fixed (defined by the sequential problem) and you add more parallel processors to *arrive at the solution faster*.
- The serial fraction in a code becomes the dominant factor in this model. *Serial fraction is hard to determine in general.*
- Achieving good strong scaling speedup for a large scale system with thousands of processors is **very hard**.
- Strong scaling analysis makes sense for systems with few processors and negligible communication overhead → shared memory.

$$S_p = \frac{T_s}{T_p} = \frac{1}{f + \frac{1-f}{p}}$$

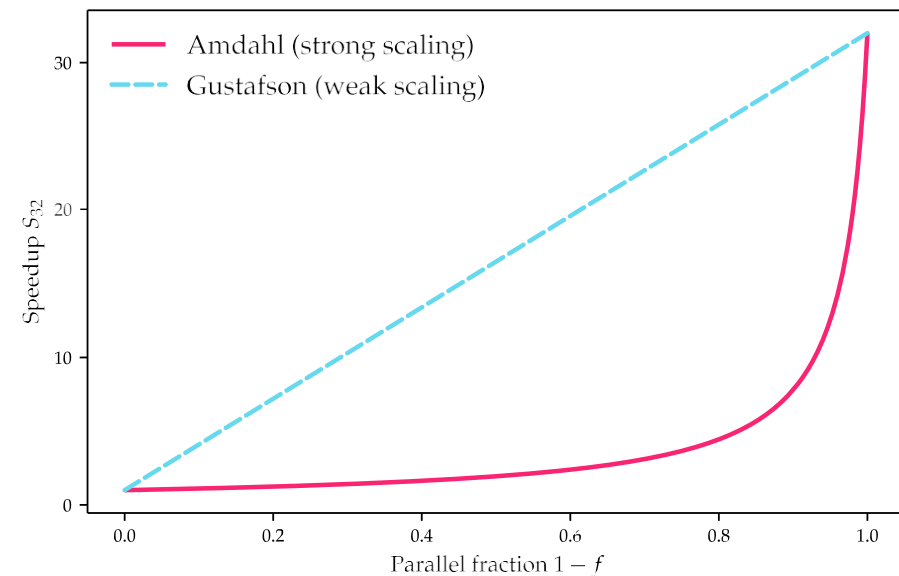
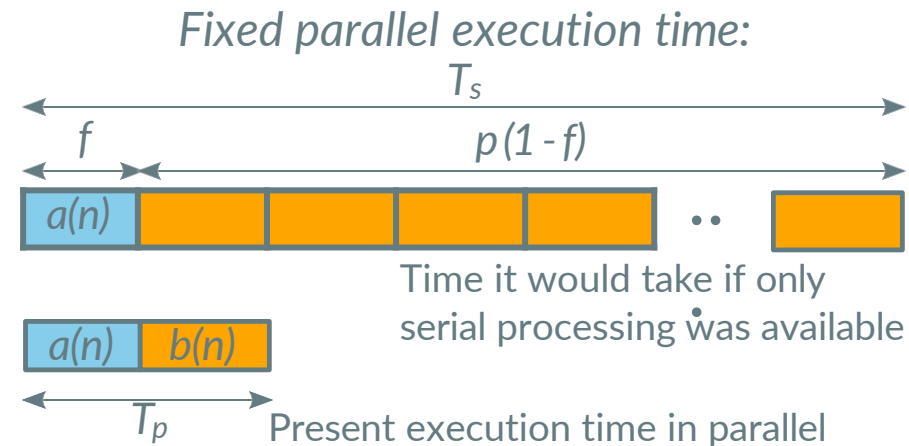


WEAK SCALING

- A speedup analysis formulated with this model is called **weak scaling**.
- It assumes the execution time per processor (the work per processor) is fixed and you add more parallel processors to increase the problem size to arrive at a larger solution *in the same time* as it would take for one processor working on a p -times smaller problem.
- The argument is equivalent to a work size of the parallel problem that is p times the serial problem, i.e., $W_p = pW_s$. The speedup is

$$S_p = \frac{W_p/T_p}{W_s/T_s} = \frac{pT_s}{T_p} = \frac{pT_s}{T_s + T_o}$$

where this assumes a diminishing serial fraction for $n \rightarrow \infty$ but non-zero parallel overhead as $T_o = T_o(n; p)$.



Observable speedup for 32 processors as a function of the parallel fraction in a code, expressed in terms of Amdahl's Law and Gustafson's Law.